

Module 4: Supervised Learning
Submodule: 4.8 Kernels
Lecture: Designing Kernels

XCS229 Machine Learning

[Tengu Ma]

Contents

I	Introduction	1
II	Understanding Kernels	1
	Test Time Computation	2
III	Kernel Matrix and Validity	2
	Kernel Design Workflow	3
	Kernel Trick	3
IV	Examples of Kernel Functions	3
	Interpretation of Kernel Functions	4
V	Summary	5

I. Introduction

In this lecture, we explore the concept of kernels in machine learning. The focus will be on how to compute the inner product of features and the implications of this for algorithms. We start by understanding that we can often define our methods directly in terms of the kernel function rather than the feature mapping.

II. Understanding Kernels

We start with the observation that the inner product of features is all that matters for certain algorithms. Thus, the focus can shift from defining the feature mapping ϕ to defining the kernel function K . The core idea is that

we can work directly with the kernel function without needing to know the explicit form of ϕ .

We do not need the explicit form of ϕ as long as K is a valid kernel. However, it is essential to ensure that the kernel function satisfies certain properties so that the algorithm can be effective.

Test Time Computation

At test time, when given a new data point x we compute $\theta^T \phi(x)$ as:

$$\begin{aligned}\theta^T \phi(x) &= \left(\sum_i^n \beta \phi(x^{(i)}) \right)^T \phi(x) \\ &= \sum_i^n \beta_i K(x^{(i)}, x)\end{aligned}$$

If the kernel function K can be computed efficiently, then we do not need to compute ϕ explicitly. Note that $x^{(i)}$ is from the training set, so at test time we still need to look at the training examples. Typically you only need to store θ and the training examples can be ignored, but not here.

If you design a good $\phi \iff$ designing a good K

III. Kernel Matrix and Validity

We start with the notation for data points and kernel matrices:

$x^{(1)}, \dots, x^{(n)}$ data points

Let $K \in \mathbb{R}^{n \times n}$ be the kernel matrix defined as:

$$K_{ij} = K(x^{(i)}, x^{(j)})$$

Having K represent both a matrix (the LHS) as well as a function (the RHS) can be confusing, so this is worth noting.

Theorem 1. *K is a valid kernel function if and only if for all $x^{(1)}, \dots, x^{(n)}$, the kernel matrix K is positive semi-definite (PSD).*

The proof can be approached in two directions:

- To show that if the kernel K is valid, then the corresponding kernel matrix is PSD follows from the definition:

$$\langle \phi(x), \phi(z) \rangle \implies \text{the matrix is PSD}$$

- The reverse direction requires more effort and careful consideration given the regularity conditions required for the kernel functions involved.

Kernel Design Workflow

The typical workflow for creating a kernel function is:

1. Design the kernel function K .
2. Verify that the kernel is valid by checking whether the corresponding kernel matrix K is PSD (from theorem 1) or by constructing $\phi(\cdot)$.
3. Run the algorithm (called Kernel trick or kernelized) .

Kernel Trick

The kernel trick allows us to transform algorithms operating in the feature space in a way that we can use kernel functions instead of featuring mappings directly. This can be applied to various contexts, such as starting with linear regression or logistic regression and translating those algorithms into the kernel space.

The kernel trick means that an algorithm developed with feature mapping $\phi(x)$ can instead be expressed with kernel K .

Not all algorithms can be transformed using the kernel trick. For example, some algorithms, such as those using L1 regularization, may not be easily kernelized since they do not only rely on pairwise inner products.

The key requirement for kernelization is that the algorithm or function can be expressed as the inner products between pairs of data points. If an entire algorithm can be expressed as an operation solely relying on such pairs, it can potentially be kernelized.

While we primarily discussed one kernel, several other kernels can be created for specific applications. Some examples include polynomial kernels, Gaussian kernels, and other custom kernels designed based on the problem at hand.

IV. Examples of Kernel Functions

We can define several kernel functions, including:

$$K(x, z) = (\langle x, z \rangle + c)^k.$$

For any choices of c and k , this is a valid kernel. For example, with

$k = 2$, we have the feature mapping:

$$\phi(x) = \begin{bmatrix} c \\ \sqrt{2cx_1} \\ \sqrt{2cx_2} \\ \sqrt{2cx_3} \\ \vdots \\ x_1^2 \\ \vdots \\ x_d^2 \end{bmatrix},$$

A common kernel function is the Gaussian kernel defined as:

$$\begin{aligned} K(x, z) &= \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right) \\ &= \langle \phi(x), \phi(z) \rangle \end{aligned}$$

For this kernel, $\phi(x)$ and $\phi(z)$ represent features that can become very complex and may require infinite dimensions to fully express.

Interpretation of Kernel Functions

- The inner product between two feature vectors can be understood as a measure of similarity between them. When x and z are similar, the kernel $K(x, z)$ gives a larger value, thus serving as a similarity metric.
- The kernel trick enables one to avoid directly computing infinite-dimensional features by replacing the inner product calculations with evaluations of the kernel function. This allows easier computations without explicitly defining the high-dimensional feature space.

However, it is essential to recognize the computational implications:

- The runtime complexity increases to $O(n^2)$ due to pairwise calculations, where n is the number of data points.
- This becomes prohibitive for very large datasets, as n^2 can grow very rapidly (e.g., for $n = 1,000,000$, it results in 10^{12} calculations).

Another significant limitation is that kernel methods often depend on handcrafted features, which require intentional design and can limit the method's effectiveness. Conversely, newer approaches in machine learning, like deep learning, automatically learn features from data. For a model:

$$\theta^T \phi_w(x),$$

where $\phi_w(x)$ is parameterized based on learned data features, demonstrates that modern methods often introduce learnable features that adapt to the dataset.

Finally, while kernel methods provide a powerful way to handle non-linear classification and regression, their dependence on handcrafted features and computational efficiency in large datasets has led to a shift towards learned feature representations in modern machine learning contexts.

V. Summary

In this lecture, we explored the concept of kernels in machine learning, focusing on their role in facilitated computation through inner products of features without explicitly knowing the feature mappings. Here are the key takeaways:

- **Kernels Over Feature Mappings:** The essential idea behind kernels is that algorithms can often be defined directly through kernel functions rather than the explicit feature mappings ϕ .
- **Validity of Kernels:** A kernel K is deemed valid if it is possible to express it in terms of inner products of mappings ϕ . The kernel matrix must be positive semi-definite (PSD).
- **Kernel Trick:** The kernel trick enables transforming algorithms to work in a higher-dimensional space without computing the features directly, aiding complex computations efficiently.
- **Examples of Kernels:** Various kernels, such as polynomial kernels and Gaussian (RBF) kernels, illustrate the versatility in kernel design. Each kernel function may correspond to a specific type of feature mapping and problem domain.
- **Computational Implications:** The substitution of explicit feature mapping for kernel functions generally increases computational overhead (to $O(n^2)$), which might become prohibitive for large datasets.
- **Shift Towards Learned Features:** The trend in modern machine learning is toward automatic feature learning (as seen in deep learning), which contrasts with earlier methods that relied heavily on handcrafted features through kernels.